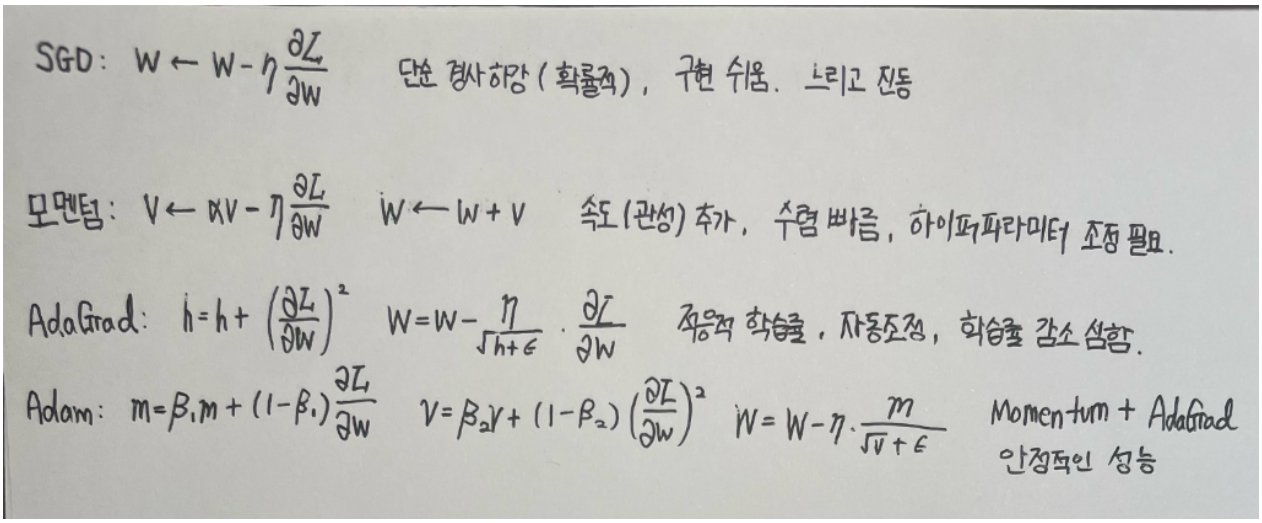
6장. 학습 관련 기술들(1) 실습문제

신경망

학번: 202404252 이름: 원종우

※ hwp 파일 한 개, py 파일 3개 제출하세요.

1. 6.1 절에서 소개한 매개변수 업데이트 방법 4가지의 이름, 수식, 특징을 간단히 적으시오.



※ C 드라이브 아래 Neural_Net 폴더 아래, ch06 폴더를 만들고 아래 문제 2번, 3번 코드를 작성하시오. 교재 Github(https://github.com/WegraLee/deep-learning-from-scratch/tree/master)에서 필요한 파일을 다운로드하여 적절한 폴더에 넣으시오.

2. 6.1.7 절의 optimizer_compare_naive.py 코드를 다음과 같이 수정하여 실행하시오.

- 목적 함수를 $f(x,y) = \frac{1}{10}x^2 + y^2$로 수정하시오.
- 초깃값(initial value)을 (-5, 1)로 변경하시오.
- 50번 반복하시오.
- optimizer마다 반복 후, optimizer 이름과 그 x_history와 y_history 값을 적절히 출력하시오.

3. 6.1.8 절의 optimizer_compare_mnist.py 코드를, 100개 노드를 가지는 은닉층을 3개를 두도록 수정하여 Visual Studio에서 실행하시오. 어느 optimizer가 좋은지 판단하시오. 왜 좋은가?

아담이 가장 좋다. Adam은 모멘텀과 적응적 학습률 조정(AdaGrad의 장점)을 결합한 방식으로,
손실이 가장 빠르고 안정적으로 줄어들며, 오버슈팅이나 진동이 적다.
따라서 아담이 MNIST 분류 문제에서 손실률이 가장 낮고 수렴 속도가 가장 빠른 optimizer이다.

4. (팀플 관련) Neural_Net 폴더 아래 data 폴더를 만들고 작성하시오.
(1) https://github.com/zalandoresearch/fashion-mnist/tree/master 에서 data/fashion/ 폴더에서 4개의 압축 파일을 다운받으시오.
(2) Neural_Net/data/fashion/ 에 (1)에서 다운로드한 4개 파일을 넣으시오.
(3) 위 사이트에서 mnist_reader.py 파일을 다운로드하고 data 폴더 안에 넣으시오
(4) optimizer_compare_mnist.py 파일을 fashion mnist 데이터에 대해 실행하는 것으로 코드를 수정하여 optimizer_compare_fashion_mnist.py에 저장하여 제출하시오.

```
import sys, os
import matplotlib.pyplot as plt

sys.path.append(os.path.join(os.path.dirname(__file__), '..'))
```

```
from data.fashion import mnist_reader
from util import smooth_curve
from multi_layer_net import MultiLayerNet
from optimizer import *

x_train,     t_train     =     mnist_reader.load_mnist('../data/fashion/',
kind='train')
x_test, t_test = mnist_reader.load_mnist('../data/fashion/', kind='t10k')

train_size = x_train.shape[0]
batch_size = 128
max_iterations = 2000

# 1. 실험용 설정==========
optimizers = {}
optimizers['SGD'] = SGD()
optimizers['Momentum'] = Momentum()
optimizers['AdaGrad'] = AdaGrad()
optimizers['Adam'] = Adam()
# optimizers['RMSprop'] = RMSprop()

networks = {}
train_loss = {}
for key in optimizers.keys():
 networks[key] = MultiLayerNet(
 input_size=784, hidden_size_list=[100, 100, 100, 100],
 output_size=10)
 train_loss[key] = []

# 2. 훈련 시작==========
for i in range(max_iterations):
 batch_mask = np.random.choice(train_size, batch_size)
 x_batch = x_train[batch_mask]
 t_batch = t_train[batch_mask]

 for key in optimizers.keys():
 grads = networks[key].gradient(x_batch, t_batch)
 optimizers[key].update(networks[key].params, grads)

 loss = networks[key].loss(x_batch, t_batch)
 train_loss[key].append(loss)

 if i % 100 == 0:
 print("===========" + "iteration:" + str(i) + "===========")
```

```
 for key in optimizers.keys():
 loss = networks[key].loss(x_batch, t_batch)
 print(key + ":" + str(loss))

# 3. 그래프 그리기==========
markers = {"SGD": "o", "Momentum": "x", "AdaGrad": "s", "Adam": "D"}
x = np.arange(max_iterations)
for key in optimizers.keys():
 plt.plot(x,       smooth_curve(train_loss[key]),       marker=markers[key],
markevery=100, label=key)
plt.xlabel("iterations")
plt.ylabel("loss")
plt.ylim(0, 1)
plt.legend()
plt.show()
```